第六章 集合类型、生命周期

6.1 集合类型——动态数组、键值对（KV）

- 动态数组：Vector，元素尺寸一样，但可以动态增减数量
  - 用来存储数量不确定的相同元素集合，访问时间复杂度O(n)
- 键值对（KV）：HashMap，每一个key都有唯一的Value与之配对
  - 建立数据索引，访问时间复杂度O(1)~O(n)

6.1.1动态数组Vector：Vec<T> 类型

- 创建Vector：
  - 1、Vec::new 调用Vec中的new关联函数

```
let v: Vec<i32> = Vec::new();
```

    - 关键点：显式地声明了类型 Vec<i32>，即都是存的i32类型的数据
  - 2、vec![] 是一种宏定义创建数组，创建时给予初始化值：

```
let v = vec![1, 2, 3];
```

    - 关键点：编译器通过初始化的值确定数组元素类型，即i32
- 数据存入Vector：
  - push方法：v.push(125); 【定义时必须是mut】

动态数组Vector：Vec<T> 类型

- 读取元素：
  - 1、通过下标索引访问 [ ]

```rust
let v = vec![1, 2, 3, 4, 5];
let x = v[0];  // Copy 类型：复制值
println!("{}", x);

let v_str = vec![String::from("a"), String::from("b")];
// let s = v_str[0];  // 错误！不能移动所有权
let s_ref = &v_str[0];  // 正确：借用
println!("{}", s_ref);
```

思考：这里所有权的转移为什么是错误的？

如果允许 let s = v_str[0];，那么：
1. s 获得了第一个 String 的所有权
2. v_str[0] 现在变成了无效数据（悬挂指针）
3. v_str 的其他部分仍然有效，但第一个元素无效

这会导致向量处于不一致的状态，可能引起内存安全问题

  - 2、使用 get 方法 .get()

动态数组Vector：Vec<T> 类型

- 读取元素：
  - 2、使用 get 方法 .get()

```
let v = vec![1, 2, 3, 4, 5];

let third: &i32 = &v[2];
println!("第三个元素是 {}", third);

match v.get(2) {
    Some(third) => println!("第三个元素是 {third}"),
    None => println!("去你的第三个元素，根本没有！"),
}
```

  - v.get(2) 访问第三个元素，但返回了 Option<&T>

动态数组Vector：Vec<T> 类型

- 读取元素：
  - 下标索引的访问方式更简便，但有可能数组越界导致报错（编译或运行时）
  - .get()方法访问复杂一些，但避免了数组越界

```
let v = vec![1, 2, 3, 4, 5];

let does_not_exist = &v[100];
let does_not_exist = v.get(100);
```

6.1.2 KV 存储 HashMap

- HashMap （哈希映射）中存储的是一一映射的 Key-Value 键值对，通过key可以直接找到对应的Value

HashMap 像一排智能储物柜

```
索引:   0     1     2     3     4
      ┌─────┬─────┬─────┬─────┬─────┐
      | 空  | 空  | 空  | 空  | 空  |
      └─────┴─────┴─────┴─────┴─────┘
```

插入第一对数据： `"Alice" -> 95`

1. 计算哈希：`hash("Alice") = 142`
2. 计算索引：`142 % 5 = 2` （假设有5个桶）
3. 放入桶2：

```
text

索引:   0     1     2     3     4
      ┌─────┬─────┬─────┬─────┬─────┐
      |     |     |Alice|     |     |
      |     |     |  →  |     |     |
      |     |     | 95  |     |     |
      └─────┴─────┴─────┴─────┴─────┘
```

插入第二对数据： `"Bob" -> 88`

1. `hash("Bob") = 238`
2. `238 % 5 = 3`
3. 放入桶3：

```
text

索引:   0     1     2     3     4
      ┌─────┬─────┬─────┬─────┬─────┐
      |     |     |Alice| Bob |     |
      |     |     |  →  |  →  |     |
      |     |     | 95  | 88  |     |
      └─────┴─────┴─────┴─────┴─────┘
```

6.1.2 KV 存储 HashMap

- 创建HashMap：使用new方法创建，insert插入

```
use std::collections::HashMap;

// 创建一个HashMap，用于存储宝石种类和对应的数量
let mut my_gems = HashMap::new();

// 将宝石类型和对应的数量写入表中
my_gems.insert("红宝石", 1);
my_gems.insert("蓝宝石", 2);
my_gems.insert("河边捡的误以为是宝石的破石头", 18);
```

需要手动引入库，
类似于#include

6.1.2 KV 存储 HashMap

- 查询HashMap：使用get方法可以获取元素，返回Option<&T>

```
use std::collections::HashMap;

let mut scores = HashMap::new();

scores.insert(String::from("Blue"), 10);
scores.insert(String::from("Yellow"), 50);

let team_name = String::from("Blue");
let score: Option<&i32> = scores.get(&team_name);
```

6.1.2 KV 存储 HashMap

- 更新HashMap：使用insert，或entry配合or_insert方法更新

```rust
fn main() {
    use std::collections::HashMap;

    let mut scores = HashMap::new();

    scores.insert("Blue", 10);

    // 覆盖已有的值
    let old = scores.insert("Blue", 20);
    // 查询新插入的值
    let new = scores.get("Blue");
    // 查询Yellow对应的值，若不存在则插入新值
    let v = scores.entry("Yellow").or_insert(5);
    // 查询Yellow对应的值，若不存在则插入新值
    let v = scores.entry("Yellow").or_insert(50);
}
```

- **使用** `insert` **当**：
  - 需要强制更新值
  - 不关心旧值，或者需要获取旧值
  - 明确知道要覆盖
- **使用** `or_insert` **当**：
  - 只想在键不存在时初始化
  - 需要累加或修改现有值
  - 执行计数、聚合等操作

6.1.2 KV 存储 HashMap

- 分析：HashMap访问的时间复杂度?

6.2 生命周期

- Rust语言特有概念，生命周期，即引用类型变量的作用域，通常是由编译器自动推导和检查，但在复杂情况下需要手动进行标注。
- 主要作用：避免悬垂引用

```
{
    let r;

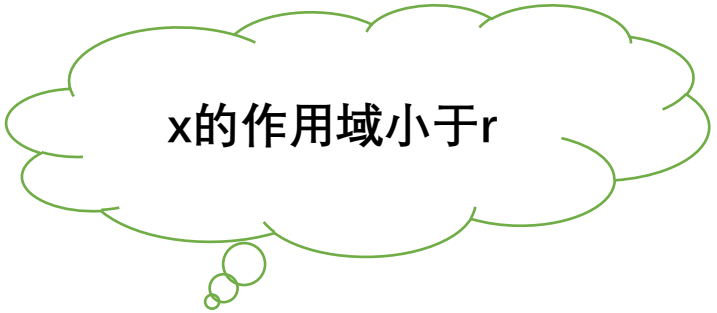
    {
        let x = 5;
        r = &x;
    }

    println!("r: {}", r);
}
```

- r 引用了内部花括号中的 x 变量，但是 x 会在内部花括号 } 处被释放，因此回到外部花括号后，r 会引用一个无效的 x

6.2 生命周期

- 上述代码的报错信息：




```
error[E0597]: `x` does not live long enough // `x` 活得不够久
  --> src/main.rs:7:17
   |
7  |                 r = &x;
   |                     ^^ borrowed value does not live long enough // 被借用的
   `x` 活得不够久
8  |             }
   |             - `x` dropped here while still borrowed // `x`
   在这里被丢弃，但是它依然还在被借用
9  |
10 |             println!("r: {}", r);
   |                                - borrow later used here // 对 `x`
   的借用在此处被使用
```

6.2 生命周期

- 借用检查：为了保证 Rust 的所有权和借用的正确性，Rust 使用了一个借用检查器(Borrow checker)，来检查我们程序的借用正确性：检查器自动判断变量的生命周期

```
{
    let r;                // ---------+-- 'a
                          //          |
    {                     //          |
        let x = 5;        // -+-- 'b  |
        r = &x;           //  |       |
    }                     // -+       |
                          //          |
    println!("r: {}", r); //          |
}                         // ---------+
```

发现 r 明明拥有生命周期‘a，但是却引用了一个小得多的生命周期‘b，会出现悬垂指针因此报错

6.2 生命周期

- 修改:

```
{
    let x = 5;                  // ----------+-- 'b
                                //           |
    let r = &x;                 // --+-- 'a  |
                                //   |       |
    println!("r: {}", r);       //   |       |
                                // --+       |
}                               // ----------+
```

x 变量只要比 r 活得久，
那么 r 就能随意引用 x 且
不会存在危险

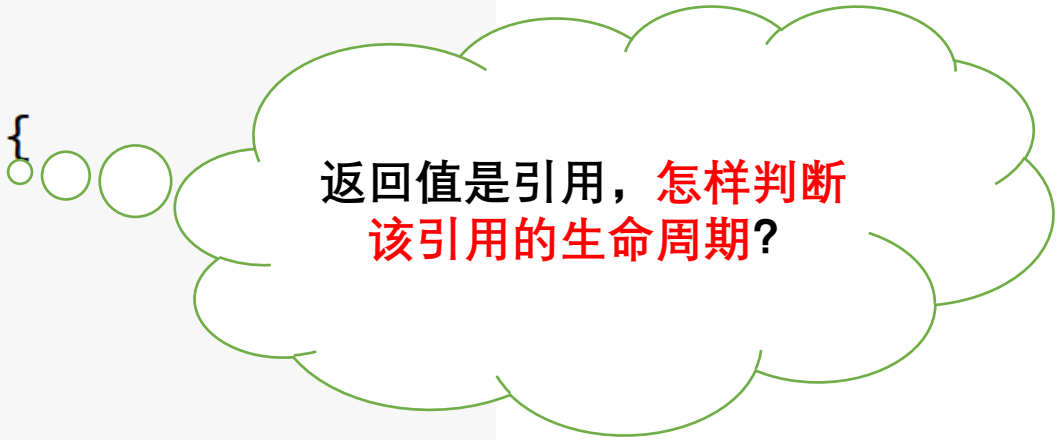
6.2 生命周期——函数中的生命周期

- 例子中，参数是两个字符串切片，返回值也是字符串切片：

```
fn main() {
    let string1 = String::from("abcd");
    let string2 = "xyz";

    let result = longest(string1.as_str(), string2);
    println!("The longest string is {}", result);
}

fn longest(x: &str, y: &str) -> &str {
    if x.len() > y.len() {
        x
    } else {
        y
    }
}
```



6.2 生命周期——函数中的生命周期

- 判断上面例子中函数返回值的生命周期，存在两种情况：
  - 返回的引用与输入的两个引用参数 x 和 y 都无关
  - 返回的引用与输入的两个引用参数 x 和 y 的任意一个或两个有关
- 两种情况的生命周期？
  - 第一种相当于在函数内部申明了一个局部变量然后返回了引用，报错！
  - 第二种输出引用的生命周期直接与输入参数挂钩，至于是哪个则需要在函数体内部进行分析！——**复杂情况下需要手动标注**

6.2 生命周期——函数中的生命周期

- 生命周期的标注语法：单引号开头，名称为单独的小写字母

```
&i32          // 一个引用
&'a i32       // 具有显式生命周期的引用
&'a mut i32 // 具有显式生命周期的可变引用

fn longest<'a>(x: &'a str, y: &'a str) -> &'a str {
    if x.len() > y.len() {
        x
    } else {
        y
    }
}
```

- 和泛型一样，使用生命周期参数，需要先声明 <'a>
- x、y 和返回值至少活得和 'a 一样久（因为返回值要么是 x，要么是 y）

6.2 生命周期——函数中的生命周期

- 注意理解：**标注**的生命周期和**实际**的生命周期没有必然联系，只是给编译检查时的一种**约束关系**。**在通过函数签名指定生命周期参数时，我们并没有改变传入引用或者返回引用的真实生命周期，而是告诉编译器当不满足此约束条件时，就拒绝编译通过。**

```
fn longest<'a>(x: &'a str, y: &'a str) -> &'a str {
```

- 这个函数头分别标注了输入参数和输出参数的生命周期，'a标注了多个输入时取值为生命周期最小的那个参数
  - 函数调用者要满足输入输出参数的约束

6.2 生命周期——函数中的生命周期

简单情况：编译器能推断

```rust
fn simple_ref(x: &i32) -> &i32 {
    x  // 编译器能推断：输入和输出生命周期相同
}
```

复杂情况：需要注解

```rust
fn longest<'a>(x: &'a str, y: &'a str) -> &'a str {
    if x.len() > y.len() { x } else { y }
}
// 编译器不知道返回的引用该绑定到 x 还是 y 的生命周期
// 'a 表示：返回值的生命周期与输入参数中较短的那个一致
```

6.2 生命周期——函数中的生命周期

- 两个例子：
- 例1：

```
fn main() {
    let string1 = String::from("long string is long");

    {
        let string2 = String::from("xyz");
        let result = longest(string1.as_str(), string2.as_str());
        println!("The longest string is {}", result);
    }
}
```

- ‘a的生命周期取值为string2的生命周期（较小的那个），result的生命周期要和string2一致，并且实际也是符合的，因此通过编译

6.2 生命周期——函数中的生命周期

- 两个例子：
- 例2：

```
fn main() {
    let string1 = String::from("long string is long");
    let result;
    {
        let string2 = String::from("xyz");
        result = longest(string1.as_str(), string2.as_str());
    }
    println!("The longest string is {}", result);
}
```

- ‘a的生命周期取值为string2的生命周期（较小的那个），result的生命周期要和string2一致，但**实际大于，因此编译报错**!

6.2 生命周期——函数中的生命周期

- 意义在哪?
  - 生命周期参数是函数与调用者之间的 **契约**，**实现分离检查原则**：**Rust 编译器在检查函数调用时，主要看函数签名（函数头），而不是函数体实现**
- 好处:
  - 封装性（用户代码只需要知道签名，不关心内部实现）
  - 编译效率（重复调用函数时不需要每次都对函数体进行检查）